Task 9: AI Pipeline Design (Armor Task)

Objective

To enhance the RAG chatbot system by implementing a robust input-output pipeline that ensures data validation, performance monitoring, and graceful failure handling for production-level reliability.

Technologies & Libraries Used

- Python
- sentence-transformers
- scikit-learn (cosine similarity)
- logging module
- re (regular expressions)
- time module

System Enhancements

- Input validation layer added
- Performance logging implemented
- Graceful error handling introduced
- Production-ready pipeline design

System Workflow

1. User inputs query
2. Input Validator checks query validity
3. Query is converted into embeddings
4. Similarity search retrieves relevant chunk
5. LLM generates response
6. Latency is recorded in log file
7. Response is returned to user

Key Features Implemented

Input Validation Rules:

• Reject queries longer than 300 characters

• Reject empty queries

• Reject queries with only special characters

Performance Logging:

• retrieval_time_ms: Time taken to fetch relevant chunk

• generation_time_ms: Time taken by LLM to generate response

• Logs stored in performance.log

Graceful Failure Handling:

• Prevents system crashes

• Returns user-friendly error message instead of traceback

Test Cases Implemented

• Valid query processing

• Long query rejection (>300 chars)

• Empty input handling

• Special character input rejection

• Performance logging verification

• API failure simulation

Errors Faced

• Validation not triggering correctly

• System processing invalid inputs

• Missing latency logs

• Unhandled exceptions causing crashes

Solutions Implemented

- Moved validation to first step in pipeline
- Added strict regex checks
- Integrated logging module
- Wrapped pipeline in try-except block

Output

```
(venv) PS D:\Daily Task\Task_9> python task9.py

  import google.generativeai as genai
🧠 Loading embedding model...
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%|┃━━━
```

```
RAG combines two main phases: **Retrieval** and **Generation**.

1.  **Retrieval Phase:**
    *   **External Knowledge Base:** You have an external, up-to-date, and authoritative knowledge base (e.g., a database of documents, web pages, company manua
ls, scientific papers, etc.). These documents are typically processed and stored in a searchable format, often as "vector embeddings" in a vector database.
    *   **User Query:** When a user asks a question, RAG first analyzes the query.
    *   **Semantic Search:** It performs a semantic search on your external knowledge base to find the most relevant pieces of information (documents, paragraph
s, or data snippets) that could help answer the user's query. This is often done by comparing the vector embedding of the query with the embeddings of the docum
ents in the knowledge base.

2.  **Augmentation Phase:**
    *   The relevant information retrieved from the external knowledge base is then "augmented" or added to the original user's prompt as additional context.
    *   The LLM receives a new, enriched prompt that looks something like:
        "Based on the following context: [retrieved relevant information], please answer the question: [original user query]."

3.  **Generation Phase:**
    *   The LLM then uses this comprehensive, context-rich prompt to generate its answer. Because it has been provided with specific, relevant, and potentially
up-to-date facts, it is much more likely to produce an accurate, grounded, and non-hallucinatory response.

**Benefits of RAG:**

*   **Reduces Hallucinations:** Grounds LLM responses in real, verifiable data.
*   **Provides Up-to-Date Information:** Allows LLMs to access current events or newly added data without needing to be retrained.
*   **Enables Domain-Specific Answers:** Allows LLMs to answer questions about proprietary data, internal documents, or highly specialized fields.
*   **Increases Transparency and Trust:** Can often provide sources or citations for the information it uses.
*   **Cost-Effective:** More efficient than constantly fine-tuning or retraining large LLMs on new data.
*   **Reduces Bias:** By sourcing information from a controlled knowledge base, it can help mitigate some biases inherent in the LLM's original training data.

In essence, RAG acts like giving a brilliant but potentially forgetful student access to an up-to-date library and specific research materials *before* asking t
hem to answer a question. This allows them to provide more accurate and well-supported answers.

💬 Ask your question (type 'exit' to quit):

📤 Response:
 Input Error: Query cannot be empty.
```

```
💬 Ask your question (type 'exit' to quit): !!!!!!

📤 Response:
Input Error: Query contains only special characters.
```

```
 💬 Ask your question (type 'exit' to quit): Machine learning is a powerful branch of artificial intelligence that enables systems to automatically learn and imp
 rove from experience without being explicitly programmed, allowing applications such as recommendation systems, natural language processing, computer vision, an
 d predictive analytics to evolve continuously based on data.

 📤 Response:
  Input Error: Query too long (Max 300 characters allowed).

 💬 Ask your question (type 'exit' to quit): exit
 👋 Exiting...
(venv) PS D:\Daily Task\Task_9>
```

Conclusion

Task 9 successfully transforms the RAG chatbot into a robust and production-grade system by adding validation, monitoring, and fault tolerance. This ensures reliability, security, and better user experience.